



Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

Ψηφιακά Συστήματα VLSI: Αναφορά 4ου Εργαστηρίου

Υλοποίηση FIR φίλτρου με AXI διεπαφή σε ZYNQ SoC FPGA

Αϊφαντή Ελισάβετ (el22821)
Χαλβατζόγλου Ιωάννα (el21906)

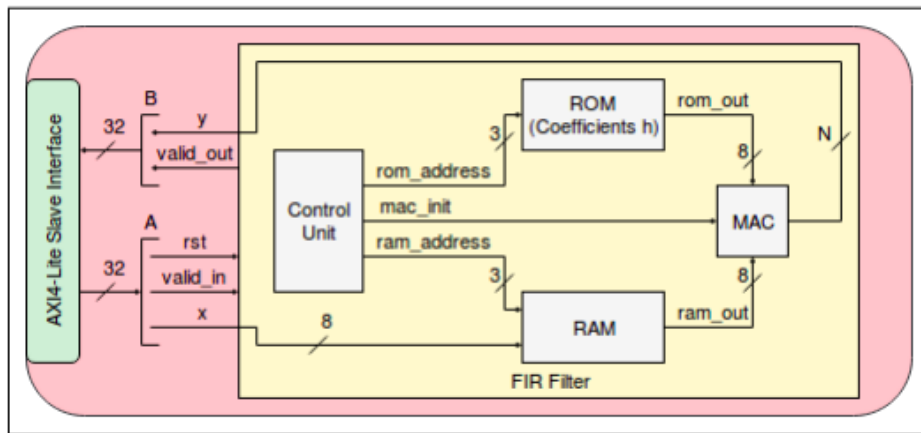
Στην παρούσα εργαστηριακή άσκηση υλοποιήσαμε:

- (1) AXI-Lite διεπαφή με σκοπό τον συγχρονισμό των σημάτων valid_in και valid_out του FIR φίλτρου, το οποίο αλληλεπιδρά με το Processing System
- (2) Εφαρμογή στο PS σε γλώσσα C για την αποστολή των δεδομένων εισόδου και τη λήψη των δεδομένων εξόδου με polling

AXI-Lite

Σκοπός της διεπαφής είναι (α) να λαμβάνει το valid_in από το PS μια φορά στην θετική ακμή του παλμού ώστε να αποσταλεί στο FIR για έναν κύκλο ρολογιού, και (β) να κάνει latching στο temp_valid_out του FIR (στο axi_valid_out) και στο αντίστοιχο αποτέλεσμα y_raw (στο y_out), ώστε να τα στείλει προς το PS για πολλαπλούς κύκλους ρολογιού μέχρι αυτό να τα διαβάσει και να μηδενίσει το axi_valid_out.

Η διεπαφή είναι σύμφωνη με την παρακάτω αρχιτεκτονική:



Εικόνα 1 : Αρχιτεκτονική FIR φίλτρου με διεπαφή AXI4-Lite.

Στην IP AXI-Lite προσθέσαμε τον παρακάτω κώδικα (με το αντίστοιχο FIR component και τα σήματα axi_valid_out, temp_valid_out, prev_valid_in, input_accepted, y_out, y_raw ορισμένα).

```
process (S_AXI_ACLK)
    variable loc_addr : std_logic_vector(OPT_MEM_ADDR_BITS downto 0);
begin
    if rising_edge(S_AXI_ACLK) then
        loc_addr := axi_araddr(ADDR_LSB + OPT_MEM_ADDR_BITS downto ADDR_LSB);

        if S_AXI_ARESETN = '0' then
            axi_valid_out <= '0';
```

```

        input_accepted <= '0';
        prev_valid_in <= '0';
        y_out <= (others => '0');
    else

        -- clear flag after cpu reads the output register
        if slv_reg_rden = '1' and loc_addr = "01" then
            axi_valid_out <= '0';
        end if;
        -- pc will send new valid_in + x after it reads axi_valid_out = 1
        -- then it gets set to 0 to prevent reading junk y values

        if temp_valid_out = '1' then
            y_out <= y_raw;
            axi_valid_out <= '1';
        end if;

        -- edge detection on previous cpu valid bit to prevent sending x twice
        prev_valid_in <= slv_reg0(8);

        if (slv_reg0(8) = '1' and prev_valid_in = '0') then
            input_accepted <= '1';
        else
            input_accepted <= '0';
        end if;

    end if;
end if;
end process;

fir_ip: fir
Port map (
    clk    => S_AXI_ACLK,
    rst    => slv_reg0(9),
    valid_in => input_accepted,
    x      => slv_reg0(7 downto 0),
    y      => y_raw,
    valid_out => temp_valid_out
);

```

Application Software

Στο παρακάτω πρόγραμμα, γίνεται αρχικοποίηση του hardware με την `init_platform()` και software reset θέτοντας high το 9ο bit της εισόδου. Στη συνέχεια, στέλνονται σειριακά τα δεδομένα x και γίνεται polling μέχρι να διαβαστεί το αντίστοιχο αποτέλεσμα. Αφού διαβαστεί, γίνεται masking για να αγνοήσουμε τα ανώτερα register bits και τυπώνεται το αποτέλεσμα στο serial terminal του Vitis.

```

#include <stdio.h>
#include <stdint.h>
#include "platform.h"
#include "xil_printf.h"
#include "xparameters.h"

```

```

#include "xil_io.h"

#define FIR_BASE XPAR_FIR_AXI_0_S00_AXI_BASEADDR
#define REG_IO_0 FIR_BASE // slv_reg0: [valid_in, x, rst]
#define REG_IO_1 FIR_BASE + 0x4 // slv_reg1: [y_out, valid_out]

int main() {

    uint8_t x_input[] = {
        208, 231, 32, 233, 161, 24, 71, 140, 245, 247,
        40, 248, 245, 124, 204, 36, 107, 234, 202, 245,
        //0, 0, 0, 0, 0, 0, 0, 0
    };

    int num_samples = sizeof(x_input) / sizeof(x_input[0]);

    init_platform();

    Xil_Out32(REG_IO_0, (1 << 9));
    Xil_Out32(REG_IO_0, (0 << 9));

    uint8_t x;
    xil_printf("Starting FIR processing...\r\n");

    for (int sample = 0; sample < num_samples; sample++) {

        x = x_input[sample];

        // edge detection in axi prevents from giving fir the same input twice (valid_in lasts one clk
        cycle)
        Xil_Out32(REG_IO_0, (1 << 8) | (x & 0xFF));
        Xil_Out32(REG_IO_0, (0 << 8) | (x & 0xFF));

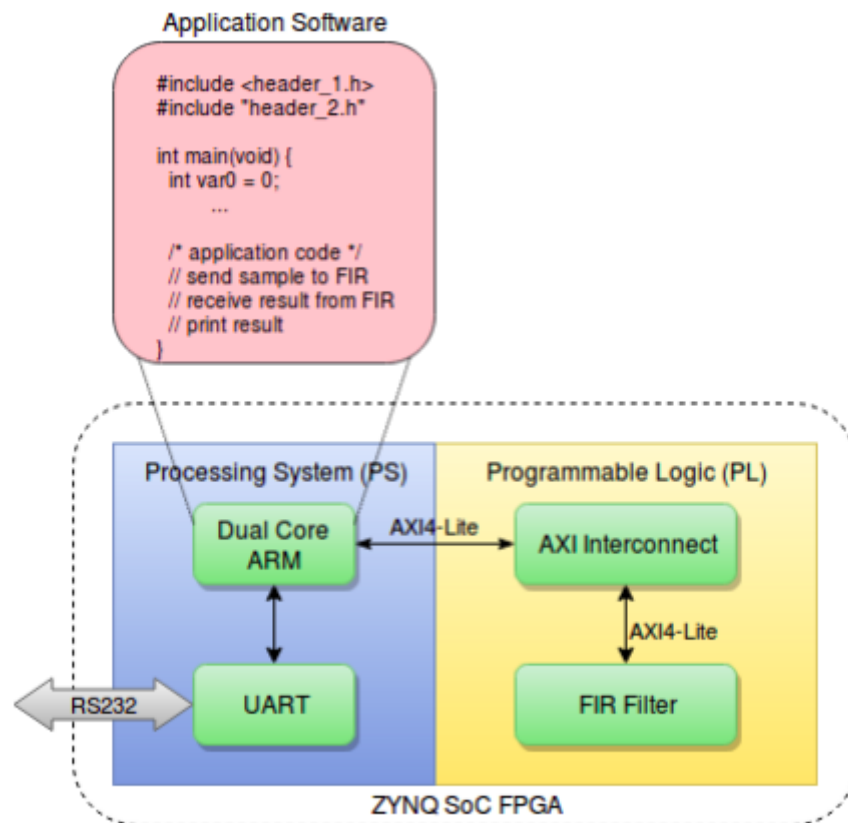
        uint32_t y_raw;
        do {
            y_raw = Xil_In32(REG_IO_1);
        } while (!(y_raw & (1 << 19)));

        int32_t y = y_raw & 0x7FFFF;
        xil_printf("y[%d]: %d\r\n", sample, y);
    }

    xil_printf("Finished FIR processing.\r\n");
    cleanup_platform();
    return 0;
}

```

Η ολική αρχιτεκτονική του συστήματος είναι ως παρακάτω:



Εικόνα 2 : Αρχιτεκτονική συστήματος.